Infinite Open-World Master Prompt v6 (Final) — Quest 3S + PC

Nov 7, 2026 · @Ari Taylor

0. Easy mode: start here

The developer's work is kept to setup once, play, press buttons, and say short phrases. Claude Code does the rest and keeps going on its own.

One-time setup (about 1 hour):

1. Install Node.js (LTS), Git, and Claude Code on the PC.
2. Enable Developer Mode on the Quest 3S (Meta Horizon phone app → device settings → Developer Mode).
3. Install Meta Quest Developer Hub on the PC (it includes adb), connect the Quest by USB, and accept the prompt in the headset.
4. Make an empty folder, download this doc as Markdown and save it there as docs/MASTER_PROMPT.md, open Claude Code in the folder, and paste the first message below.

First message to Claude Code:

```
Read docs/MASTER_PROMPT.md fully. Then: create CLAUDE.md from section 1A,
split the spec into docs/spec/NN-name.md files (one per section) so you
only read what each task needs, set up the repo, dev server, headset
bridge, tests, and bench, and finish M0. Work autonomously. Decide
anything not marked OPEN yourself and log it in docs/DECISIONS.md.
When M0 runs, tell me exactly what to do in the headset.
```

The developer's only commands:

Say this	Claude Code does
continue	Reads PROGRESS.md and NEXT.md, picks the next task, builds it
ready to test	Starts the dev server, connects the Quest, lists 3–5 simple steps

Say this	Claude Code does
check my report	Reads the newest headset reports and captures, fixes what they show
looks fake: <what>	Compares captures with references, fixes light, material, scale, or density
feels bad: <what>	Fixes comfort, controls, or stutter, then asks for a retest
here are GLBs / photos	Runs the asset intake or reference ingestion
undo that	Reverts the last change with git and explains what was undone

Guided test mode (in the headset): the wrist menu shows the current milestone's test steps one at a time, records frame data automatically, and ends with one-tap feedback buttons: Looks real, Looks fake, Stutter, Felt sick, Controls wrong, Love it. Each tap attaches a capture, so the developer never writes test reports by hand.

Safety net: Claude Code commits after every working step, tags each milestone, and never deletes saves or assets without a backup. Any change can be undone with one phrase.

Session continuity: at the end of every session Claude Code writes `docs/NEXT.md` (what's next, open questions, last test result). A new session only needs "continue". To save usage, Claude Code reads only the spec files a task needs and keeps tasks small.

Honest pacing: this is a very large game. Each milestone may take many sessions, but every milestone ends with something playable, so the idea grows step by step instead of all at once.

1. How to use + role

This is the master spec for Claude Code. Save it in the repo as docs/MASTER_PROMPT.md, put the CLAUDE.md from section 1A at the repo root, then build milestone by milestone (section 17). Claude builds nearly everything itself (code, shaders, procedural geometry, materials, tools); the developer adds their own GLBs when they have them and runs the in-headset tests. Every technology choice is already locked (section 3), so the model should implement rather than re-debate. Each milestone ends with an in-headset test on Quest 3S before moving on.

Role. You are a senior engine architect and VR technical director who ships standalone-VR open worlds in Three.js + WebXR. You write production code: deterministic, worker-threaded, data-oriented, profiled on device. You never trade Quest 3S frame stability for visual features. Where realism and performance conflict, you fake the realism (baked, cached, impostored) and keep the frame time.

Working rules for the model:

- Output real, runnable code and data schemas, not concepts.
- Every system states its CPU/GPU cost on Quest 3S and its PC scaling.
- Every generated thing is reproducible from a seed plus saved deltas.
- Ask the developer only for in-headset test results, references, or decisions marked open.

1A. Claude Code operating manual

Claude Code works autonomously in short verified loops and gets real headset data through a dev-server bridge, so it never builds blind. The developer's job shrinks to: put on the headset, play, press Report, and answer occasional decisions.

CLAUDE.md (copy to repo root):

```
# Project: infinite photoreal VR open world (Three.js + WebXR, Quest 3S + PC)
Full spec: docs/MASTER_PROMPT.md. Progress: docs/PROGRESS.md. Decisions:
docs/DECISIONS.md.

## Golden rules
- Photorealism is the #1 visual goal; 72 Hz on Quest 3S is never broken to
get it.
- Respect budgets in spec section 4. Every new system reports its cost.
- Generation is deterministic: no Math.random, Date, or platform Math in
src/gen.
- No allocations in the frame loop. Pools only.
- Every generated thing = seed + params + deltas. Never save meshes.
- Everything real is interactive and logical (spec 9A); only nearby props are
simulated.
- Real-world scale in meters (spec 5A table).
- Check Three.js APIs in node_modules/three (pinned), never from memory.
- Argue real decisions in docs/DECISIONS.md first (spec 1B).
- Follow the Three.js playbook (spec 3B); do not re-argue settled tactics.
```

```
## Commands
- npm run dev        # HTTPS dev server + headset bridge (/__report,
/__capture)
- npm run quest      # adb reverse tcp:5173 + open on headset
- npm test           # unit + determinism + coherence + save round-trip
- npm run bench      # headless perf/correctness scenes, writes
reports/bench/*.json
- npm run shots      # Playwright screenshots of reference scenes ->
reports/shots/
- npm run assets     # intake GLBs, fetch CC0, generate materials, compress,
LODs
- npm run bake       # home-PC baker for lightmaps, cubemaps, panoramas

## Definition of done (every task)
1. Types, lint, and tests pass. 2. Bench within budget on desktop proxy.
3. Screenshots reviewed. 4. PROGRESS.md updated. 5. Headset test checklist
   written to docs/HEADSET_TESTS.md for the developer.

## Ask the developer only for
Headset test results, reference photos, their GLBs, and decisions marked
OPEN.
Everything else: decide yourself, log it in docs/DECISIONS.md, keep working.
Batch questions into one message; never stop working while waiting.
End every session by updating docs/NEXT.md.
```

Repo layout:

Path	Contents
src/core	ECS, loop, pools, floating origin, job system
src/xr	WebXR session, input, controller, IK, comfort
src/gen	Deterministic generators (macro, meso, micro, interiors, assets)
src/world	Chunks, streaming, persistence, simulation
src/render	Uber shader, lighting, splats, LOD, culling, post
src/physics, src/ai, src/audio, src/ui, src/save	One system each
workers/	Generation, AI, save workers

Path	Contents
server/	Dev server bridge, baker queue, Director proxy
tools/	Asset pipeline, baker, bench and screenshot runners
assets/incoming	Developer GLBs dropped here, auto-processed
data/	Entity library, presets, biomes, materials (JSON)
docs/	Spec, progress, decisions, headset tests, licenses
reports/	Headset reports, captures, bench output (read by Claude)

Work loop (every task):

1. Read PROGRESS.md and the relevant spec section; write a short plan in the task.
2. Build the smallest vertical slice that runs.
3. Run tests, bench, and screenshots; fix until green.
4. Commit with a clear message; update PROGRESS.md and DECISIONS.md (one line per decision with the reason).
5. Write the headset checklist; wait for the developer's report before marking the milestone done.

Use subagents for independent work (asset pipeline, research, code review against budgets). Pre-commit hooks run typecheck, lint, and the fast tests. Large raw binaries go in Git LFS; generated assets are rebuilt by script, not committed.

Headset bridge (how Claude sees the Quest):

- Dev over USB: `adb reverse tcp:5173 tcp:5173`, so the Quest opens `https://localhost:5173` as a secure context; Chrome remote DevTools via `chrome://inspect`.
- **Report button** on the wrist menu: posts seed, position, settings, the last 30 seconds of frame times per system, worker lag, memory, and thermal step to `/__report`, which writes `reports/headset/<time>.json` into the repo. Claude reads it directly.

- **Capture button:** renders one frame from the head pose to an offscreen target and posts a PNG to `/__capture`, so Claude can see what the headset sees and compare it with reference photos.
- Auto-report on any hitch over 20 ms or any error, with the stack trace.

Backend (small Node server, same repo): in development it is the Vite dev server plugin; in production a small service. It runs the baker job queue, proxies the AI Director (the API key never ships to the client), and serves the baked-data cache.

1B. Self-driving protocol

Claude Code drives the project itself: it argues every real decision against its own best counterargument, settles disputes with measurements, reviews its own work adversarially, and learns from external repos the developer adds. This replaces asking the developer.

Decision protocol (argue with yourself): any choice that affects performance, look, or architecture gets a short entry in `docs/DECISIONS.md` before code is written:

```
## D-042 Static batching: BatchedMesh vs merged geometry (2026-11-07)
Options: A) BatchedMesh per chunk+material  B) mergeGeometries per chunk
For A: per-object culling, add/remove without rebuild, one draw call
For B: simplest, zero per-object overhead
Devil's advocate vs A: multi_draw may be emulated on Quest -> CPU cost
Criteria: Quest frame time, draw calls, rebuild hitch on delta change
Spike: bench_static_batching (both, 3,000 objects)
Decision: A for buildings and props, B for terrain and road surfaces
Revisit if: multi_draw unsupported on Quest Browser or rebuild > 1 ms
```

Rules: at least two options; a devil's-advocate paragraph attacking the favorite; measurable criteria; a spike benchmark when uncertain; a revisit trigger. Decisions marked OPEN in this spec go to the developer only if the spike cannot settle them.

Measurements are the judge: `npm run bench` scenes produce numbers on the desktop proxy; headset reports (1A bridge) are the final truth. A decision that loses on numbers is reversed, logged, and noted in NEXT.md.

Ground truth for Three.js: the pinned version in `node_modules/three/src` and `node_modules/three/examples/jsm` is the authority, not memory. Before using any Three.js API, Claude Code checks it exists in that version; before upgrading Three.js, it

reads the migration guide, upgrades on a branch, and runs the full test and bench suite. Upgrades happen only between milestones.

Adversarial self-review: after each task, a reviewer subagent checks the diff against the golden rules: budgets, determinism, frame-loop allocations, disposal of GPU resources, scale, and photoreal intent. It must find at least one concrete improvement or explicitly sign off with reasons.

External repo ingestion (the developer will add Three.js repos):

1. Repos go into `vendor-ref/<name>` (reference only, not imported directly).
2. Claude Code writes `docs/repos/<name>.md`: what it does, license, Three.js version, maintenance state, techniques worth taking, Quest risks, and a verdict: **adopt** (dependency), **adapt** (port the technique into our code), **learn** (ideas only), or **reject**.
3. Never copy code under an incompatible license; prefer porting techniques into our own modules behind an interface.
4. Anything adopted gets a before-and-after bench and a DECISIONS.md entry.
5. A running `docs/TECHNIQUES.md` index lists every technique learned and where it is used, so knowledge from each repo compounds.

2. Core vision and non-negotiables

Build an infinite, persistent, self-learning open world for VR: drive, explore, and fight on foot in a world that is generated from real-world logic and remembers everything. Photorealism is the number-one visual goal of the whole game: every system, asset, and shader is designed to produce photographic output, then made fast. Quest 3S reaches it through captured and precomputed realism (section 13A) at a locked 72 Hz; a PC-rendered tier streamed to the Quest pushes it further at 90 Hz.

Locked creative decisions:

- **Setting:** near-future (about 2040) grounded realism. Real-world infrastructure and decay as the base, with a cyberpunk layer (neon, dense signage, augmented ads, drones) that intensifies in dense urban cores and fades in rural areas. Realism always wins over style.
- **Core loop:** free-roam sandbox with emergent jobs (section 15). No scripted campaign in v1.
- **Players:** single-player v1. Architecture stays multiplayer-ready (server-authoritative-friendly data, deterministic generation).

Non-negotiables:

1. Never a pre-made map. The world is generated from seeds, real-world placement logic, and the learned player model.
2. Full persistence: return to any place and it is identical, plus every change the player made.
3. Any environment can exist: megacity, decayed industrial, freeway, forest, desert, snow, coast, rural, interiors, underground.
4. One world serves driving, exploration, and FPS combat. Layout adapts to playstyle without splitting into separate worlds.
5. The world gets more personal and less repetitive the longer someone plays, across sessions.
6. Comfort and stable frame time are never sacrificed. No forced camera motion, ever.

Realism pillars (what "photoreal" means on a mobile VR chip): true photorealism at 72 Hz stereo is not possible on Quest 3S, so the game wins by removing the things that make VR look fake. Every system is judged against these six pillars:

1. **Correct scale:** every object at real-world size, matched to the player's calibrated eye height (section 5A table).
2. **Correct light:** physical light units, believable exposure, and white balance by time of day (section 13).
3. **Correct materials:** calibrated PBR values from real references, with wear driven by age and use.
4. **Image stability:** no shimmering, no popping, no visible tiling. In VR these break realism more than low polygon counts.
5. **Correct density and logic:** the right amount of stuff, in the right places, for the right reasons (section 5).
6. **Correct motion and sound:** physics, animation, and audio that react the way real things do.

3. Locked technology decisions

Every choice below is picked for Quest 3S first, with PC as a scaled-up tier of the same code. Do not swap these without a profiled reason.

Area	Choice	Why
Language / build	TypeScript, Vite, ES modules	Type safety across workers; fast HMR for headset iteration
Renderer	Three.js WebGLRenderer (WebGL2) with one GLSL uber shader on every tier for v1; WebGPURenderer re-tested at M12 (section 3B)	WebGL2 is the proven WebXR path in Quest Browser; one shader codebase for all tiers
XR	WebXR via Three.js XRManager, fixed foveation level 1–2, WebXR Layers for UI/text quads	Layers keep text sharp without extra eye-buffer cost
Architecture	ECS with bitECS (struct-of-arrays)	Cache-friendly, cheap iteration, easy to serialize and network later
Physics	Rapier3D (WASM, compat build) on main thread, fixed 72 Hz step	Fast, deterministic, good vehicle raycasts; main thread keeps hand latency lowest
Threads	3 workers: generation+meshing, navigation+AI planning, save/IO	Main thread only renders, inputs, physics, gameplay
Worker transport	Transferable ArrayBuffers; SharedArrayBuffer where COOP/COEP headers allow	Zero-copy geometry handoff
RNG / hashing	PCG32 seeded from xxHash32(worldSeed, chunkX, chunkZ, layer)	Every chunk regenerates identically, in any order
Noise	Seeded simplex/fBm + domain warp, run in worker	Terrain, biome, and density fields
Navigation	recast-navigation-js (WASM) per chunk tile, built in worker	Real navmeshes for NPCs and enemies
Assets	glTF 2.0 + Meshopt, textures KTX2 (ETC1S albedo, UASTC normals), via gltf-transform	Smallest GPU memory and download on Quest
Saves	IndexedDB (idb wrapper), binary records via msgpackr + fflate	Compact, async, survives sessions

Area	Choice	Why
Audio	Web Audio API; HRTF PannerNode for nearest 8 sources, equal-power for the rest	HRTF is expensive; pool it
Profiling	OVR Metrics Tool, Chrome remote DevTools, in-game perf HUD	Frame time truth comes from the device
Desktop dev	Immersive Web Emulator + flat-screen debug camera	Fast iteration between headset tests
Multiplayer (later)	Authoritative Node server, WebSocket first, WebTransport later	Deterministic world means only deltas sync

Rendering scale rule: WebXR framebuffer scale is fixed per session. Quality adapts at runtime through LOD distances, scatter density, shadow cascade reach, and particle counts, never by changing eye-buffer resolution mid-session.

Photoreal rendering tiers (same world data, same code base):

Tier	Where pixels are computed	Realism target	Frame rate
Quest Captured	Quest 3S, displaying cloud-baked lighting, photogrammetry, Gaussian splats, and baked horizon panoramas	About 75–85% photoreal in good conditions, higher for distant views	72 Hz locked
PC Streamed	Gaming PC (RTX 4070-class or better) running the PC renderer, streamed to Quest via Link, Air Link, or Virtual Desktop	About 85–90%+ photoreal	90 Hz
PC Flat	Same PC renderer on a monitor	Same as PC Streamed	60–144 Hz

The Quest Captured tier works the way a photo does: the expensive light simulation happens ahead of time on a GPU, and the headset only displays the result. Realism percentages are honest targets to validate with the reference-match test (section 16), not guarantees.

3A. Engine performance rules

These rules prevent the hitches and precision bugs that ruin infinite VR worlds. They apply to every system, on every platform.

- **Floating origin:** float32 breaks down past about 10 km from the origin (jitter, z-fighting). Rebase the world origin every 1 km during a fade-free frame; store world positions as Float64 chunk coordinates plus local offsets.
- **Zero allocation in the frame loop:** preallocated vectors, matrices, and pools for bullets, decals, particles, and audio voices. Garbage collection pauses are the most common VR hitch in JavaScript.
- **Shader warmup:** compile every material variant behind the loading fade with `renderer.compileAsync`; new variants at runtime are forbidden.
- **GPU upload budget:** at most 2 MB of geometry and texture uploads and at most 2 new meshes per frame, queued and time-sliced.
- **Worker job system:** jobs are cancellable, prioritized by the predictive agent, and time-sliced to 6 ms per worker slice so results arrive steadily.
- **Deterministic math in generation:** generation uses its own deterministic functions (no `Math.sin`, `Math.random`, or `Date`), so every device and future multiplayer peer builds identical chunks.
- **Occlusion culling:** frustum culling per sub-tile, distance culling per object class, portal-cell culling for interiors, baked potentially visible sets per 32 m cell, and a coarse software depth buffer (128 × 64) in a worker as the fallback (section 3B).
- **Unified surface types:** one surface ID per material (asphalt, concrete, metal, glass, wood, dirt, grass, sand, snow, water, flesh) drives footsteps, tire grip, bullet penetration, decals, particles, and impact sounds.
- **Asset caching and offline play:** a service worker caches all assets with the Cache API; the game installs as a PWA and runs offline after first load. Download budget: under 400 MB for Quest's first install, with streamed biome packs.
- **Distribution:** ship as a PWA in the Meta Horizon Store and on the web. Because everything is data-driven, a native port stays possible later if WebXR overhead blocks a target.

3B. Three.js playbook (every tactic, argued)

Each tactic below was argued against its strongest alternative; the losing option and why it lost are kept so Claude Code does not re-litigate them. Verify each API against the pinned Three.js version (section 1B) and confirm the uncertain ones in the M0 spike.

Renderer and architecture:

Tactic	Decision	Rejected alternative and why
Renderer	WebGLRenderer (WebGL2) on every tier for v1, one GLSL shader codebase	WebGPURenderer + TSL everywhere: elegant single source, but its WebXR, multiview, and Quest performance are less proven; revisit at M12 only if the PC tier needs compute shaders
Framework	Vanilla Three.js driven by bitECS	React Three Fiber: nicer ergonomics, but reconciliation and per-frame allocations cost frame time we cannot spare
Scene graph	Flat; static objects `matrixAutoUpdate = false`, `updateMatrix()` once; ECS writes transforms	Deep hierarchies with auto updates: traversal cost grows with world size
Render loop	`renderer.setAnimationLoop` only (required for XR); no `scene.traverse` per frame	requestAnimationFrame: does not run inside XR sessions
Render thread	Main thread renders (WebXR is unavailable in workers); everything else in workers	OffscreenCanvas rendering in a worker: incompatible with WebXR

WebXR on Quest:

Tactic	Decision
Stereo	Multiview (`multiviewStereo: true` or the pinned version's equivalent) to submit both eyes in one pass; verify MSAA works with it in the M0 spike

Tactic	Decision
Anti-aliasing	`antialias: true` (4x MSAA in the XR layer) + `alphaToCoverage` on foliage, fences, and hair
Foveation	`renderer.xr.setFoveation(1.0)` on Quest Captured; lower on PC Streamed
Frame rate	`session.updateTargetFrameRate(72)` for Quest Captured; 90 on PC Streamed; chosen at session start, never switched mid-play (switching judders)
Resolution	`renderer.xr.setFramebufferScaleFactor` before the session per tier; never changed mid-session
Reference space	`local-floor`; recenter on seated calibration
UI text	WebXR Layers (`XRQuadLayer`, cylinder layers) for wrist menu and dashboards so text is sampled once and stays sharp
Poses	Read controller and hand poses inside the XR frame callback (already predicted for display time); physics hand targets use that frame's pose
Controller and hand models	Own photoreal hand models; the default model factories fetch from a CDN, so cache or replace them for offline play

Draw calls and batching (the biggest CPU lever):

Tactic	Used for
`BatchedMesh` per chunk and material	Unique static geometry: buildings, facades, props, road furniture. Per-object frustum culling and visibility, add and remove without rebuilding, one draw call
`InstancedMesh` per 32 m sub-tile	Many identical objects: trees, poles, bollards, windows, debris. Sub-tile split gives frustum culling; a worker sorts by distance and sets `count` for distance culling

Tactic	Used for
`mergeGeometries`	Terrain patches and road surfaces that never change per object
Shader-only instancing (`gl_InstanceID` + noise in the vertex shader)	Grass and ground cover: positions computed on the GPU, zero CPU instance data
Texture arrays (`CompressedArrayTexture` from KTX2) + a per-vertex layer index	One material for a whole kit, so different-looking objects still batch together
Material passes	Exactly four material families: opaque, alpha-to-coverage cutout, transparent (glass, water), additive emissive. Everything maps to one of them

Shaders:

Tactic	Decision and reason
Material base	Custom `UberMaterial` (ShaderMaterial assembled from selected Three.js ShaderChunks plus our own), not MeshStandardMaterial: we pay only for the lighting we use (sun, SH probe volume, lightmap, 4 points)
Variants	Features as `defines` ; `customProgramCacheKey` controls variants; every variant precompiled with `renderer.compileAsync` behind the loading fade
Precision	`mediump` in fragment shaders where safe (Adreno runs FP16 roughly twice as fast); safe because of camera-relative positions and the floating origin
Branching	No dynamic branches on per-pixel data; use defines, `step` , and `mix`
Fog	Analytic height fog and aerial perspective in the uber shader (built-in fog is too simple)
Tonemapping	`AgXToneMapping` applied in-material (no post pass on Quest); `ColorManagement` enabled, output `SRGBColorSpace`

Tactic	Decision and reason
Depth range	Near 0.05 m, far 4 km, 24-bit depth; reverse-Z when the pinned version and device support it. Never `logarithmicDepthBuffer` on Quest: writing fragment depth disables early-Z and costs heavily on tile GPUs. Beyond 3 km the horizon is a panorama with no depth
Uploads	`renderer.initTexture()` and staged geometry uploads within the 3A budget, so first sight of an object never hitches

Lighting and shadows:

Tactic	Decision and reason
Shadowmask (key photoreal trick)	Static world never renders into the shadow map. The baker stores baked indirect light plus a sun-visibility mask; only dynamic objects (cars, NPCs, the player's body, moved props) cast real-time shadows. Final sun light = sun × min(baked visibility, real-time shadow). Film-quality static shadows plus correct dynamic shadows, with no double darkening, at a fraction of the cost
Shadow map	One directional light, 1024^2 on Quest, texel-snapped to stop shimmer, updated only when dynamic casters move; PC uses 3 cascades
Probes	SH L1 probe grid per chunk stored in a `Data3DTexture`, sampled trilinearly in the uber shader for dynamic objects and splats
Reflections	Box-projected cubemaps per block (custom chunk), prefiltered by roughness mip; `PMREMGenerator` only for the sky environment
Lightmaps	Second UV set from xatlas; RGBM decoded in the uber shader and blended between time keys

Culling:

Tactic	Decision and reason
Baked PVS	The baker also computes potentially visible sets per 32 m cell (which blocks and interiors can be seen from there). Free at runtime, never late, ideal for dense cities
Portals	Interiors use room-portal culling
Software occlusion	128 × 64 depth raster in a worker as the fallback before PVS is baked
Rejected: hardware occlusion queries	Results arrive 1–2 frames late, and fast VR head turns make objects pop in. Kept only as a debug tool

LOD, impostors, decals, particles:

- LOD switching with a stereo-consistent dithered crossfade over 0.3 s via a per-instance fade attribute; the built-in `LOD` object is not used because it hard-swaps.
- Octahedral impostors baked by the asset pipeline for trees, buildings, and vehicles; lit by probes.
- Decals: persistent hits are instanced quads with `polygonOffset` from a decal atlas; road wear is baked into a per-chunk detail texture. `DecalGeometry` is avoided at runtime (CPU heavy).
- Particles are stateless: instanced quads whose motion is computed in the vertex shader from spawn time and seed, so the CPU never updates them.

Streaming, memory, uploads:

- `GLTFLoader` + `MeshoptDecoder` + `KTX2Loader` (transcodes to ASTC on Quest, BC7 on PC, in its worker pool).
- Two texture tiers per material: a 128^2 version always resident, the full version loaded near the player, swapped when ready.
- Every chunk eviction calls `dispose()` on its geometries and unique textures; a leak check compares `renderer.info.memory` before and after a 5 km round trip.
- Geometry is built in workers as typed arrays, transferred, and wrapped in `BufferGeometry` on the main thread within the upload budget.

Animation, text, queries:

- Animation LOD: full NPCs update their `AnimationMixer` at 72 Hz within 10 m, 36 Hz to 30 m, then switch to the crowd's vertex-animation textures.
- Skeletons capped at 64 bones on Quest; shared animation clips across all humans.
- Dynamic in-world text (debug, UI) uses troika-three-text SDF; district signage is baked into atlases (5B).
- Gameplay raycasts (bullets, ground, grabs) go through Rapier; three-mesh-bvh `acceleratedRaycast` is used for precise visual queries and the baker. Plain `Raycaster` on large meshes is banned.

PC tier effects, an honest caveat: screen-space post-processing inside WebXR is fragile with WebGLRenderer. PC Streamed realism therefore comes mainly from higher-resolution baked data, more real-time lights and shadow cascades, denser geometry, and in-material effects. Post-processing (pmndrs `postprocessing` with N8AO) runs on PC Flat; XR post is an M12 spike, together with re-testing WebGPURenderer.

Library set (pin versions; verify maintenance and license before adding anything else):

Library	Role
three (pinned)	Renderer, loaders, XR
bitecs	ECS
@dimforge/rapier3d-compat	Physics
three-mesh-bvh	BVH raycasts and GPU ray traversal for the baker
three-gpu-pathtracer	Baker cubemaps and panoramas
recast-navigation (+ its three helpers)	Navmeshes
@gltf-transform/core, cli, functions + meshoptimizer	Asset pipeline, LODs, compression
KTX-Software (toktx / basisu)	KTX2 encoding in tools
xatlas (WASM build)	Lightmap UVs
A Gaussian splat renderer (Spark or GaussianSplats3D)	Splats; chosen by an XR spike
troika-three-text	SDF text

Library	Role
postprocessing + n8ao	PC Flat effects
idb, msgpackr, fflate	Saves
vitest, playwright	Tests, benches, screenshots
lil-gui (dev only)	Live tuning on desktop

4. Quest 3S and PC performance budgets

Quest 3S runs at a locked 72 Hz (13.9 ms per frame) with headroom for thermals; PC VR targets 90 Hz and flat PC 60–144 Hz. These are starting targets: verify every milestone on device and tighten where it throttles.

Budget	Quest 3S	PC tier
Main-thread CPU total	⩽ 8 ms	⩽ 6 ms at 90 Hz
Render submit	⩽ 4 ms	⩽ 3 ms
Physics (Rapier)	⩽ 2 ms	⩽ 2 ms
Gameplay + ECS	⩽ 1.5 ms	⩽ 1.5 ms
VR controller IK + hands	⩽ 0.5 ms	⩽ 0.5 ms
Draw calls	⩽ 120 (target 90)	⩽ 1,500
Triangles on screen	⩽ 400k	⩽ 4M
GPU texture memory	⩽ 400 MB	⩽ 3 GB
Geometry memory	⩽ 150 MB	⩽ 1 GB
JS heap	⩽ 600 MB	⩽ 2 GB
Dynamic lights	1 sun + 4 point near player	1 sun + 32 clustered
Shadows	1 cascade, 1024^2, 35 m	3 cascades, 2048^2, 250 m
Full-screen post	none (tonemap + fog in material shaders)	bloom, SSAO, SSR, TAA

Budget	Quest 3S	PC tier
Particles	⩽ 2,000	⩽ 50,000
Texture max size	1024² hero, 512² props, 2048² terrain atlas	4096²

Rules that keep Quest stable:

- Use OVR_multiview2 when the Three.js version supports it (verify on current release), so stereo costs one draw submission.
- No MSAA above 4x; no transparency sorting of large surfaces; alpha-test foliage, never alpha-blend.
- Everything static is InstancedMesh or merged per chunk and material. One material per kit, texture arrays or atlases per biome.
- Thermal guard: if frame time exceeds 12.5 ms for 2 seconds, drop one quality step (density, LOD reach, shadow distance). Raise again only after 20 seconds stable.

LOD distances (Quest; PC × 1.75):

Level	Range	Content
LOD0	0–40 m	Full mesh, interactable, physics colliders
LOD1	40–120 m	Simplified mesh, no interiors, static colliders only
LOD2	120–400 m	Merged block meshes, baked window emissive
Impostor	400 m–3 km	Octahedral impostors and skyline cards
Horizon	3 km+	Terrain silhouette mesh + skybox layers

Photoreal budgets (Quest Captured tier): baked lightmaps ⩽ 128 MB resident, reflection cubemaps ⩽ 48 MB, horizon panoramas ⩽ 32 MB, visible Gaussian splats starting at about 400k (tune on device), sorted off the main thread.

5. World knowledge layer and generation logic

The generator models how real places come to exist, using general real-world knowledge, not only supplied references. References only calibrate proportions, palettes, and materials per region. Everything is placed by cause and effect, then biased by the player

model.

Generation runs in three scales, coarse to fine, all seeded and deterministic:

1. **Macro (region, 4 km cells):** elevation, rivers, coastline, climate (latitude, altitude, moisture), and terrain-first zoning. Steep ground becomes park or forest, flat land near water becomes port and industry, valleys get freeways and rail, plateaus get suburbs.
2. **Meso (district, 512 m cells):** road hierarchy (freeway, arterial, collector, local), interchanges, rail lines, district type, district age, and wealth gradient.
3. **Micro (chunk, 128 m):** parcels, buildings, props, vegetation scatter, decals, NPC spawn points. Interiors generate on demand when a door is opened or the player is within 15 m.

Entity library. A data-driven catalog of real-world things, each with real proportions, materials, footprint rules, and placement logic. Categories: residential, commercial, industrial, civic (hospitals, schools, police, fire), religious, transport (stations, airports, ports, depots, parking), utilities (power plants, substations, pylons, water towers, treatment plants, cell towers), infrastructure (bridges, tunnels, retaining walls, culverts), vegetation by climate, vehicles, street furniture, and fauna.

```
{
  "id": "gas_station",
  "footprint": { "w": [30, 45], "d": [25, 40] },
  "requires": ["road:arterial|freeway_exit"],
  "prefers": ["intersection_corner", "near:freeway_ramp<300m"],
  "avoids": ["district:cbd_core", "same_type<800m"],
  "ageVariants": ["new", "worn", "abandoned"],
  "interior": "convenience_store_small",
  "combatTags": ["cover:pumps", "cover:vehicles", "verticality:none"]
}
```

Placement logic (cause and effect, not random scatter):

- Power: plants feed substations, substations feed neighborhoods via pylons and poles. Every lit building is reachable on the grid graph.
- Commerce follows traffic: gas stations at exits and corners, malls at arterial junctions, corner shops at local intersections.
- Industry follows logistics: ports need coast plus rail; factories need rail or freeway; scrapyards cluster near industry and freeways.

- Wealth follows views and distance from industry; decay follows lost industry and freeway cuts.
- Civic services are spaced by population: one fire station per roughly 2 km radius, hospitals near arterials, schools inside residential districts.
- Water drains downhill; bridges appear where roads cross rivers; tunnels where grades exceed 8%.

History layer. Each district has a founding era and events (boom, decline, renewal, disaster). This drives architecture mix, building age, maintenance state, vacancy, graffiti density, and construction sites. Old cores sit beside new towers; this contrast is what makes places feel lived-in.

Interior grammar. Building type sets a room program. Apartments get entry, kitchen, living room, bedrooms, bath. Offices get lobby, cores, open floors, meeting rooms. Hospitals get wards on double-loaded corridors. Factories get floor lines, mezzanine offices, loading docks. Rooms are laid out by a constrained BSP or graph solver, then furnished from the entity library by room tag.

Player-model bias. The learned model (section 8) changes probabilities only inside what the logic allows. A player who loves rooftops gets more accessible rooftops; a gas station never appears in the middle of a forest without a road.

5A. Terrain, vegetation, buildings, interiors at full fidelity

This is where generated worlds usually look fake: terrain without erosion, roads that float, trees in random noise, and buildings that are boxes. Each subsystem here follows how the real thing forms.

Terrain:

- Heightfield per region from fBm plus domain warp, then hydraulic and thermal erosion run once per 4 km region in a worker and cached. Erosion is what makes slopes, gullies, and river valleys read as real.
- Rivers follow flow accumulation downhill; lakes fill basins; coastlines get beaches on gentle slopes and cliffs on steep ones.
- Rendering uses CDLOD with vertex geomorphing, so terrain never pops.
- Roads carve the terrain: cut and fill with embankments at 30°, retaining walls where slopes exceed that, culverts where streams cross, and a blend zone of gravel and dirt at road edges.

- Terrain materials: up to 8 splat layers per chunk from texture arrays, selected by slope, height, moisture, and wear, plus a macro variation texture and cheap stochastic sampling near the camera to remove visible tiling.

Vegetation:

- Placement follows ecology: species by climate, altitude, slope, and moisture; Poisson-disk spacing; competition (big trees suppress nearby saplings); dense edges along roads, rivers, and clearings; and nothing on steep rock or road cuts.
- Each species has 3 meshes (hero, mid, card) plus an impostor; transitions use dithered crossfade, never hard swaps.
- Ground cover (grass, weeds, litter, snow patches) is GPU-instanced in a 30 m radius around the player and fades by distance.
- Seasons change foliage color and density by latitude and the in-game calendar (autumn color, bare winter trees, snow).

Buildings:

- A facade shape grammar: footprint → massing (setbacks, podium, tower) → floors → bays → windows, doors, balconies, AC units, signage, and roof equipment.
- Style libraries per region and era (Chinese high-rise tile and glass, US brick industrial, 1960s concrete, 2040 glass and LED).
- Wear masks driven by building age and district maintenance: streaks under windows, rust, stains, broken windows, graffiti, and vines.
- Landmark generator: each district gets 1–3 unique hero structures (a supertall, a stadium, a temple, a bridge) built from rarer grammar rules, used for navigation and memory.

Interiors:

- Portal-cell culling per room and floor; only the current and adjacent rooms are rendered.
- Windows show the exterior through the real world where open, or an impostor where not.
- Everything sensible is interactive: light switches control room lights, doors and drawers open, taps run, TVs and screens play, appliances hum, elevators move between floors.
- Skyscrapers: elevators and stairwells load floors on demand; rooftops connect to helipads, cranes, and neighboring roofs for traversal.

- Room lighting uses per-room probes plus one shadowless point light for the room's main fixture.

Real-world scale standards (VR realism depends on these):

Element	Size
Door	0.9 × 2.1 m
Residential ceiling	2.6–2.8 m
Commercial floor-to-floor	3.6–4.2 m
Stair rise / run	17–18 cm / 28 cm
Handrail height	0.9–1.0 m
Table / counter height	0.75 m / 0.9 m
Curb height	15 cm
Lane width	3.5 m urban, 3.75 m freeway
Sidewalk	2–4 m
Car (sedan)	4.8 × 1.85 × 1.45 m
Street light	8–12 m tall, every 30–40 m
Player eye height	measured at calibration, never assumed

5B. World simulation and AI Director (the intelligence layer)

The world is smart because it runs as a living simulation behind the generator, checks its own logic, and can call a language model for writing. Nothing in this layer ever runs in the frame loop.

Region simulation (AI worker, 1 tick per game minute):

- Coarse model per 512 m district: population, wealth, faction control, traffic flow, crime, maintenance, construction projects, and economy (jobs, prices).
- Player actions feed it: clearing a gang site shifts territory, repeated crashes on a road bring roadworks, spending in a district raises its maintenance.

- **Time passes while you are away:** when a chunk is revisited, the simulation applies elapsed time first (construction progresses, vegetation regrows in abandoned lots, wrecks get towed, graffiti spreads, shops open or close). Persistence plus change is what makes a world feel alive rather than frozen.

Coherence validator (runs after micro generation, before meshing):

- Hard checks: every door reachable, no road dead-ends without a reason (cul-de-sac, barrier, construction), no building on a road or river, every powered building on the grid graph, every interior has an exit, NPC navmesh connected.
- Soft checks scored against the realism rules: a gas station without a road, a skyscraper in a forest, or a supermarket with no parking scores low and is regenerated with the next seed variant.
- Failures are logged with chunk seed for the debug tools.

AI Director (optional cloud language model, cached):

- Writes the text a real place is full of: shop names, signage, graffiti tags, billboard copy, street and district names, NPC dialogue barks, job briefings, radio host chatter, and lore notes found in buildings.
- Called asynchronously from a background queue (for example the Claude API) with the district's simulated state as context. Results are validated (length, language, content filter), saved into chunk data, and never regenerated, so places keep their identity.
- Offline or rate-limited: fall back to a seeded template grammar with region-specific word banks. The game is fully playable with the Director off.
- Sign text is rendered into texture atlases per district in a worker, never as live text meshes.

Privacy: the player model and telemetry stay on the device. Only district context (no personal data) is sent to the Director, and it can be disabled in settings.

6. Biomes, city presets, road modules

These presets come from the developer's references (Shenzhen, Guangzhou, Detroit, West Virginia, coastal and snow photos) and are extended by the world knowledge layer to cover any place. Each preset is a data file: palette, materials, density curves, prop sets, and combat tags.

City presets:

Preset	Reference	Block size	Key rules
Asian superblock CBD	Shenzhen Futian	400–500 m superblocks	8–10 lane boulevards, central park axis, one supertall landmark per district, height falloff from core, elevated ring expressway, construction hoardings, flags, haze
Dense night market district	Shenzhen street, Mong Kok, Shinjuku	60–120 m	Lantern canopies, stacked signage, escalators and footbridges, planters as cover, warm red + cyan palette
US decay grid	Detroit Milwaukee Junction	100 × 200 m	Sunken freeway trenches, 40–60% parcel occupancy, vacant lots, scrapyards, sawtooth factories, overgrown parcels, diagonal rail lines
Waterfront promenade	Zhuhai, Shenzhen bay	linear	Palm rows, flower beds, stone balustrades, glass towers set back, water to the horizon
Cultural park	Shenzhen temples and gardens	campus	Pagoda and tile-roof kit, walled courtyards, stone stairs, lakes with pavilions, bonsai and rock gardens

Biomes (each sets vegetation, ground materials, water type, weather odds, fog, and architecture style):

Subtropical urban hills, temperate Appalachian forest gorge, coastal tropical beach, dry coastal scrub hill, snow highland, desert, river valley farmland, and rail-cut pine forest. Water uses one shader with per-biome color, turbidity, and wave parameters: harbor murk, calm bay, brown estuary, turquoise reef, green river, mirror lake.

Road modules (spline-following, each with a fixed cross-section spec):

Module	Cross-section and rules
Sunken freeway	3–4 lanes each way, 3 m shoulders, Jersey median, retaining wall one side, 30° grass embankment other side, surface overpass every 200–400 m
Elevated expressway	Box girder on columns every 30–40 m, 2–3 lanes each way, noise walls, ramps at 4–6% grade
At-grade arterial	4–10 lanes, hedge or planted median, signalized intersections, bus bays
Local street	2 lanes, parking both sides, sidewalks 2–4 m, street trees
Ramp	Lane splits at shoulder width, 6% max grade, merge taper 150 m
Interchange	Cloverleaf (loop radius 80–100 m), diamond, or stack, chosen by road classes and space
Rail line	Ballast bed, catenary or diesel, girder bridges with stone piers over rivers, diagonal cuts across grids
Rural road	2 lanes, gravel shoulders, guardrails on drops, ditches

Connectors between biomes: arch bridges over gorges, rail bridges over rivers, tunnels through ridges, causeways over water, and overpass stacks. Transitions are placed where terrain justifies them, never as hard seams.

Surface realism (where most realism comes from): decal scatter rules per module for cracks, asphalt patches, oil stains, tire marks, trash, leaves, dirt runoff at shoulder-grass edges, and graffiti on walls facing traffic.

6A. Reference and texture ingestion pipeline

The developer supplies photos, Google Maps street and aerial screenshots, and textures over time. Each kind of input becomes a specific data file, so references improve the generator instead of sitting in a folder.

Inputs and what they become:

Input	Extracted	Becomes
Aerial / satellite screenshot	Block size, road hierarchy, interchange type, parcel occupancy, green ratio	City preset and road module parameters
Street View screenshot	Cross-section (lanes, shoulders, walls, medians), prop spacing, sign gantries, wear	Road module spec and decal scatter rules
Skyline / landscape photo	Height distribution, silhouette, haze, palette, vegetation type	District height curves, fog and color grading per biome
Night street photo	Light colors, emissive density, sign stacking, reflections	Night lighting preset and signage density
Close-up surface photo	Albedo, color variation, wear pattern	PBR material (pipeline below)
Interior photo	Room program, furniture, clutter density	Interior grammar and furnishing sets

Photo → PBR material pipeline (offline build step, Node + scripts):

1. De-light: remove shadows and highlights so albedo is lighting-free.
2. Make tileable with offset blending or texture synthesis.
3. Calibrate albedo into physical ranges (most materials 0.04–0.8 linear reflectance; no pure black or white).
4. Derive height and normal maps, roughness from albedo variance plus a material-class table, and ambient occlusion from height.
5. Pack occlusion, roughness, and metalness into one ORM texture.
6. Encode KTX2 (ETC1S albedo, UASTC normal and ORM) with mipmaps, and generate a Quest size and a PC size.
7. Register in the material library with its surface type (section 3A) and biome tags.

Free photogrammetry and PBR sources to fill gaps: Poly Haven and ambientCG (CC0 textures, HDRIs, and models). Every asset passes the same calibration and budget checks.

Validation: a material preview scene renders each new material under 4 standard lights (noon, overcast, dusk, night street) on desktop and in the headset before it ships.

6B. Asset strategy: Claude builds it, GLBs plug in

Claude generates most of the world in code: geometry, materials, and lighting. CC0 libraries fill realism gaps by script, and the developer's GLBs drop in through an automatic intake. Every asset class has a procedural fallback, so the game never blocks on missing art.

What Claude generates procedurally:

Asset class	Method	Photoreal source of detail
Terrain	Noise + erosion (5A)	Scanned CC0 ground materials, macro variation
Roads, curbs, barriers, bridges	Spline sweeps of cross-section profiles	Wear decals, scanned asphalt and concrete
Buildings and facades	Shape grammar (5A) with modular trims	Scanned brick, concrete, glass; wear masks by age
Interiors and furniture	Room grammar + parametric furniture (bevelled boxes, lathes, extrusions)	Scanned wood, fabric, metal materials
Street props (poles, signs, benches, AC units, bins, gantries)	Parametric generators per type	Rust and grime masks, real dimensions
Trees and plants	Space colonization for trunks and branches, leaf and needle cards	CC0 bark and leaf atlases, species by climate
Rocks and cliffs	Noise-displaced, eroded meshes, baked to low poly with normal maps	Scanned rock materials, splat kits
Materials	Procedural PBR generator (noise, wear, dirt, edge damage), baked to KTX2	Calibrated against CC0 scans and developer photos
Sky and horizon	Atmosphere model + baked panoramas	CC0 HDRIs
Sound	Procedural layers + CC0 sound libraries	Recorded ambiences where available

Scripted CC0 fetch: `npm run assets` downloads textures, HDRIs, and models from the Poly Haven and ambientCG APIs by tag, calibrates and compresses them (6A pipeline), and records source and license in `docs/LICENSES.md` automatically.

Developer GLB intake: drop a file in `assets/incoming/` with an optional sidecar `name.json` (type, tags, surface type). The pipeline then:

1. Validates scale (meters), Y-up, origin at base center, PBR materials, and triangle count; it fixes what it can and reports the rest.
2. Generates LODs with meshoptimizer simplification, plus an impostor for far distances.
3. Bakes ambient occlusion, builds a simple collider (convex parts or boxes), and assigns grab points by type.
4. Compresses with Meshopt and KTX2, then registers the asset in the entity library with its tags, so the generator can place it immediately.

Honest hard parts, where GLBs help most: photoreal humans, hero cars, and weapons are the hardest to generate in code. Claude ships parametric versions (modular human bodies on a standard rig, car bodies from profile curves, weapons from part kits) so everything works, and replaces them with the developer's GLBs or licensed models as they arrive. Characters use a standard humanoid rig so any compatible model and animation set retargets automatically.

7. Persistence, chunks, saves

The world is saved as seeds plus parameters plus deltas, never as meshes. A visited chunk costs about 1–4 KB on disk unless the player changed it; a heavily fought-over chunk stays under about 40 KB.

Chunk system:

- Chunks are 128 m squares with 4 × 4 sub-tiles of 32 m. Vertical stacking (interiors, tunnels, underground) uses layered sub-chunks keyed by floor index.
- Key: `(worldSeed, cx, cz, layer)` hashed with xxHash32. Spatial index is a hash map plus a quadtree for range queries.
- States: `unknown → planned (macro/meso only) → generated (micro data) → meshed → resident → dormant (data only) → evicted`.
- Residency on Quest: full detail within 2 rings (25 chunks), LOD1–2 out to 4 rings, impostors beyond. The predictive agent (section 8) pre-generates along the predicted path.
- Eviction is LRU weighted by distance, revisit likelihood from the player model, and unsaved changes. Changed chunks flush to IndexedDB before eviction.

Saved per chunk:

```
interface ChunkRecord {
  v: number;                 // schema version
  key: [number, number, number]; // cx, cz, layer
  genHash: number;           // hash of generator version + params used
  params: { biome: number; district: number; age: number; wealth: number;
density: number; combatBias: number };
  deltas: Delta[];           // only what differs from regeneration
  visits: { first: number; last: number; count: number; secondsSpent: number
};
}
type Delta =
  | { t: 'remove'; entity: number }                      // destroyed or
picked up
  | { t: 'move'; entity: number; pos: Vec3; rot: Quat }  // physics props,
doors
  | { t: 'state'; entity: number; s: number }            // door open, light
broken, car burned
  | { t: 'add'; type: number; pos: Vec3; rot: Quat; data?: Uint8Array } //
dropped items, barricades, bodies
  | { t: 'decal'; type: number; pos: Vec3; normal: Vec3; scale: number }; //
bullet holes, scorch, blood, skid marks
```

Rules:

- Entity ids are deterministic (chunk key + spawn index), so deltas re-attach after regeneration.
- Decals cap at 256 per chunk; the oldest merge into a baked grime layer instead of vanishing.
- Bodies and wrecks persist as low-cost static props after 60 seconds.
- Generator versioning: if `genHash` changes, the old chunk is kept as-is for visited chunks (stored as a frozen params snapshot) so updates never rewrite places the player already knows.
- Global save: player state, inventory, vehicles owned (with positions), world clock, weather seed, and the player model, written every 30 seconds and on pause or exit. Keep 3 rotating save slots against corruption.
- Load-time target on Quest: under 6 seconds to first frame, with the nearest 9 chunks generated before fade-in.

8. Self-learning player model and predictive agent

The learner is interpretable by design: named features updated with moving averages, plus a contextual bandit that learns which content choices keep this player engaged. No black-box neural net, so every bias can be inspected and tuned. Total size is under 8 KB.

Player model (two timescales):

```
interface PlayerModel {
  v: number;
  features: Record<FeatureId, { session: number; lifetime: number }>; // EMA,
0..1
  fatigue: Record<ContentTag, number>;     // recent exposure, decays over
~20 min
  bandit: Record<DecisionId, { alpha: Float32Array; beta: Float32Array }>; //
Thompson sampling per option
  history: { biomeSeconds: Float32Array; districtSeconds: Float32Array;
kills: number; distanceDriven: number; sessions: number };
}
```

Features (about 40), grouped as: driving (avg speed, speed variance, off-road ratio, freeway vs local share, crash rate, scenic stops); exploration (interiors entered, vertical climbs, rooftop time, landmark detours, revisits, photo-mode use); combat (engagement rate, avg engagement distance, cover usage, flanking, aggression, weapon class shares, headshot ratio, retreats); and mood (night vs day time, weather preference, dense vs empty time).

Update rules:

- Every 2 seconds, observed signals update session EMAs (α 0.05) and lifetime EMAs (α 0.002).
- Fatigue rises when a content tag is seen and decays exponentially; high fatigue suppresses that tag regardless of preference. This is the anti-repetition engine.
- Bandit decisions (district type next, POI type, encounter style, transition timing) are rewarded by engagement: time spent near the content, interaction, return visits, and not driving straight past. Rewards arrive 60–300 seconds after placement.

How it changes the world (concrete):

Signal	Generator response
High rooftop time	More roof access, fire escapes, rooftop POIs

Signal	Generator response
Long freeway drives at speed	Longer sweeping roads, rarer intersections, scenic overlooks, rest stops
Close-range combat preference	Tighter streets, more interiors and doors, shorter sightlines
Long-range combat preference	Open lots, elevated perches, longer sightlines
Repeated night play	Later default time and more neon district density near the player
Lingering at landmarks	More unique landmarks per km², rarer filler
High fatigue on a tag	That tag drops out for the next stretch, even if liked

Cold start (first 10 minutes): a curated starting region (city edge with freeway, decay district, and forest gorge within 2 km) samples every playstyle. Priors are neutral but slightly favor variety, so early play teaches the model fast.

Predictive agent. Runs in the AI worker at 4 Hz, about 0.3 ms per tick.

1. Predict the path: current velocity plus road graph, plus likely destinations scored by the player model (landmarks in view, unexplored directions, preferred district types).
2. Build a priority queue of chunks along the top 3 predicted paths, weighted by time-to-arrival. At 30 m/s the agent must stay at least 8 seconds ahead.
3. Plan transitions: decide where the next biome or district change happens, so terrain and roads lead into it naturally (a ridge before a forest, an industrial edge before a decay district).
4. Plan encounters and points of interest using the bandit, placed where the player will arrive in 30–120 seconds.
5. Report generation lag to the failure system (section 16).

Balance rules (no overreaction):

- **Repetition, not single events:** a preference counts only after it repeats across at least 3 separate visits or 2 sessions. One tattoo session changes nothing world-wide.
- **Narrow effect:** a learned interest raises only its own content type (a liked tattoo shop makes tattoo shops slightly more likely nearby), never the theme of the world.

- **Hard caps:** any single tag can change its spawn frequency by at most ±25% from the world-logic baseline, and never past what real-world placement rules allow (section 5).
- **Baseline wins:** the world's realism (districts, zoning, shop mix) always comes from world logic first; the player model only nudges inside it.
- **Decay:** interests not refreshed by play fade back to neutral over about 5 sessions.

9. VR player controller

The goal is the most physical VR controller that still fits 0.5 ms of CPU on Quest 3S: physics hands that respect the world, a full IK body, and comfort options for every locomotion style. The developer has an existing controller; refactor it toward this spec rather than rewriting blindly.

Physics hands:

- Each hand is a Rapier dynamic rigid body driven toward the tracked controller pose by a PD controller (linear and angular velocity targets), not teleported. Walls and tables stop the hand.
- When the physics hand is more than 5 cm from the tracked pose, show a translucent ghost hand at the true pose; beyond 40 cm, snap the physics hand back.
- Hand colliders: palm box plus 3 capsules (thumb, index, other fingers). Fingers animate from grip, trigger, and thumb-rest touch on controllers, and from XRHand joints under hand tracking, using the same rig.
- Held-object weight: follow strength scales with object mass. Heavy objects lag and need two hands; light objects feel instant.
- Throwing uses hand velocity averaged over the last 5 physics frames plus angular velocity from the wrist.

Grab system:

- Grab points are typed: handle, knob, edge, surface, weapon grip, foregrip, magazine, steering wheel, ladder rung, ledge.
- Doors, drawers, levers, and valves are Rapier joints (revolute or prismatic) with limits and friction, moved by the hand body.
- Two-hand objects (rifles, wheel, crates) solve orientation from both hands.

IK body:

- Height and arm span calibrated on first launch (stand tall, T-pose for 2 seconds).

- Arms: two-bone IK with elbow hint from hand orientation and body side.
- Spine and pelvis: estimated from head height, head pitch, and hand positions; crouch and lean read naturally.
- Legs: procedural stepping when the pelvis moves more than 25 cm from the feet; foot IK raycasts against ground and stairs.
- The player sees their own body when looking down and in mirrors and reflections; the head mesh is hidden in first person.
- IK runs at 72 Hz for hands and arms, 36 Hz for legs.

Body collision and comfort:

- A capsule follows the head in room scale; walls push the capsule, never the camera.
- If the head enters geometry, fade to black over 0.2 s instead of letting the player see inside walls.
- Locomotion options: smooth stick (default, with speed vignette), snap turn 30° or 45°, smooth turn with adjustable speed, teleport, and an arm-swing mode. Seated and standing modes with height offset.
- Physical crouch and prone are read from real head height; a crouch toggle exists for seated play.
- Climbing: grabbing a ledge or rung makes the hand an anchor and the body moves opposite to hand motion.
- Haptics: short pulses on grab, impact, recoil, and engine rumble, scaled by intensity, via the Gamepad haptic actuator.

Input map (Quest controllers):

Action	Input
Grab	Grip
Use / fire	Trigger
Move	Left stick
Turn	Right stick
Jump / vault	A
Crouch toggle	Right stick click
Wrist menu	Look at left wrist + Y

Action	Input
Holster / inventory	Reach to body slot (hips, chest, back) and grip

Hand tracking mirrors this with pinch to grab, palm-up gestures for the menu, and no shooting without controllers in v1.

9A. Everything is interactive and logical

Any object a real person could pick up, open, use, wear, or break can be handled that way in the game, with hyper-detailed props up close. This stays fast because only objects near the player are simulated: everything else is static until touched.

Detail on demand (how hundreds of props stay at 72 Hz):

- Every room and shop is fully furnished, but distant props are part of the static BatchedMesh with baked lighting.
- Within 3 m of either hand or on any impact, a prop is hydrated: swapped for its interactive version with physics, grab points, and uses. It goes back to static once it sleeps and the player moves away.
- Only moved or changed items are saved, as deltas (section 7), so a shop the player rearranged stays rearranged forever.
- Containers (drawers, boxes, fridges, cabinets, bags) generate their contents only when opened, from a seed, so the world has unlimited detail without unlimited cost.
- Budgets on Quest: at most 150 awake physics bodies and 40 hydrated props per hand radius; hand-held items get 2–8k triangles and 512² textures, room props less.

Object data (every prop): real dimensions and mass, surface type (section 3A), grab points, physical state (open, on, full, broken, wet, dirty), and a list of uses.

Uses system (tool + gesture + target → effect): uses are data, not special code, so new behaviors are added as entries.

Use	Example tools	Effect
Ink / draw / write	Tattoo machine, marker, pen, chalk	Paints into the target's texture (skin, paper, wall)
Spray	Spray can, hose, extinguisher	Persistent decals or particles on surfaces

Use	Example tools	Effect
Pour / fill	Bottle, cup, kettle, fuel can	Moves liquid between containers; spills make wet decals
Cut / break	Knife, scissors, crowbar, hammer	Pre-fractured pieces, cut cloth or rope, opened boxes
Power	Switches, plugs, remotes, ignition	Turns devices, lights, screens, engines on or off
Wear	Clothing, glasses, helmets, gloves, watches	Attaches to body slots; changes appearance and protection
Consume	Food, drinks, medicine	Hand-to-mouth gesture; affects hunger, health, or state
Pay / trade	Cash, cards, phone	Exchanges items at counters; shop systems react
Assemble / repair	Tools, parts	Repairs vehicles or devices; attaches weapon parts

Worked example, a tattoo shop: the generator places it in a fitting district, and its inventory list includes tattoo machines, power supply, ink bottles, needle cartridges, gloves, stencil paper and transfer gel, a client chair, an autoclave, flash art on the walls, a mirror, and a counter with a card reader. The player can grab the machine, press the trigger, and drag the needle over their own skin. The game raycasts from the needle tip to the player's body mesh, finds the UV point, and paints into that body part's skin texture (512^2 per region on Quest). The tattoo is saved with the player and visible in mirrors and when looking down. The shop owner reacts if the player uses equipment without paying.

Inventories authored from real knowledge: Claude Code writes `data/inventories/*.json` for hundreds of shop and room types (tattoo studio, barber, pharmacy, hardware store, garage, kitchen, office, clinic, gym, bar, warehouse, and more). Each list has what a real one contains, quantities, and where items sit (shelf, counter, wall, drawer, floor). Brand names and labels come from the AI Director (5B) and are baked into label textures.

Real-life logic rules:

- Weight and size are real: a phone is one-handed, a TV needs two hands, a fridge can only be pushed or dragged.

- Carrying uses real capacity: hands, pockets, body slots, and a backpack limited by volume and weight.
- Ownership: taking from a shop without paying triggers the owner and possibly police; items in the player's safehouse are theirs.
- Shops restock over time through the region simulation (5B); dropped items stay where they fall until something logical moves them (cleaners, rain, other NPCs).
- The player's body persists too: tattoos, clothing, dirt, wetness, and injuries, saved with the player.
- Mirrors are low-resolution planar reflections, limited to one visible at a time on Quest.

Learning stays balanced: using one thing once (like a tattoo shop) must not change the world's theme. Rules are in section 8.

Systemic simulation (no special cases): interactions are not scripted one by one. Every object is built from parts with physical properties, and a small set of general rules makes any situation work: crashes, repairs, destruction, fire, water, electricity, and how people respond. The tattoo and the tire change are both just outcomes of the same rules.

Rule	What it means in play
Objects are assemblies	A car is wheels, tires, lug nuts, hub, brake, suspension, battery, bulbs, fluids, panels, glass. A door is a leaf, hinges, handle, lock. Any part with a fastener can be removed and refitted with the right tool
Parts have states	Intact, worn, dented, cracked, punctured, broken, burnt, wet, dirty, empty, dead. States change from damage, use, weather, and time, and persist
Materials have properties	Flammability, conductivity, hardness, brittleness, weight, water resistance. A wooden fence burns, a metal one conducts, glass shatters, plastic melts
Tools match fasteners and tasks	Wrench turns nuts, screwdriver turns screws, jack lifts at jack points, pump inflates, hammer dents or flattens, welder joins metal. Gestures are physical: rotate the wrench, pump the jack

Rule	What it means in play
Fluids move	Fuel, oil, coolant, water, ink, paint, blood. Containers hold volumes; pouring, leaks, and spills follow gravity and leave decals
Energy spreads	Fire spreads by material and wind on a cell grid; heat damages; water puts fire out; electricity needs an unbroken path from the grid or a battery; explosions apply impulses and damage in a radius
Structure holds or fails	Building and bridge pieces have integrity and supports; remove or destroy enough supports and the rest falls as physical pieces
Everything has consequences	Every action emits an event; NPCs, police, shops, reputation, the region simulation, and insurance-like repair costs all subscribe to events
People react both ways	NPCs respond to harm and to kindness: help, gifts, repairs done for them, returned items, and saving someone earn gratitude, favors, discounts, friendship, and named NPCs remember the player across sessions

Worked example, crash and tire change: the player hits a curb hard. The impact event damages the front-left tire (punctured), bends the rim (dented), and scuffs the bumper. The car pulls to the left and the dashboard shows pressure loss. The player opens the trunk, which generated a spare, a jack, and a lug wrench from the car's inventory list. They place the jack at the jack point (snap zone), pump the handle to lift, turn the wrench to remove four nuts, pull the wheel off, fit the spare, tighten, and lower the car. The old wheel lies where they dropped it until something logical moves it. Later, at a garage, they can buy a new tire, hammer the rim, or pay the mechanic NPC to do it.

Scope and order: the generic systems are built once; the content (parts lists per object type, material tables, tool sets) is data Claude Code authors at scale from real-world knowledge. Build order: vehicles and tools first (M10), then fire, water, electricity, and structure (M11), then the full social reaction model (M8 onward). Simulation detail follows the same distance tiers as everything else: full part simulation within about 30 m, aggregated states beyond.

10. Vehicles and physics

The player's car is a full raycast-wheel simulation at 72 Hz; traffic cars use a cheap kinematic spline follower until something hits them. This keeps one hero simulation plus up to 40 traffic cars inside the 2 ms physics budget.

Hero vehicle model:

- Rapier rigid body chassis with 4 raycast wheels, spring-damper suspension, anti-roll bars.
- Tire model: simplified Pacejka (magic formula) for longitudinal and lateral slip, with surface grip by material (dry asphalt 1.0, wet 0.7, gravel 0.6, grass 0.45, snow 0.3, ice 0.1).
- Drivetrain: engine torque curve, 6-speed gearbox (auto or manual), differential, ABS and traction control toggles.
- Weight transfer, downforce at speed, and aerodynamic drag.
- Data-driven vehicle definitions (mass, center of mass, torque curve, gear ratios, suspension, grip) for sedan, hatchback, SUV, pickup, sports car, van, truck, motorbike, and train (rail-locked).

VR driving:

- Physical steering wheel grabbed with one or two hands; wheel angle follows hand rotation around the hub, with force-limited return-to-center.
- Shifter, handbrake, indicators, horn, and door handles are grabbable. Pedals map to triggers.
- Seated calibration recenters the head to the driver seat. The cockpit is a stable visual frame, which greatly reduces sickness.
- Comfort options: horizon-locked camera on hard bumps, optional vignette on high yaw rate, and a speed-scaled peripheral fade.
- Exit and enter by opening the door with your hand; the IK body sits in the seat.

Damage:

- Body panels deform with vertex offsets from impact point and energy, capped at 6 deformation zones per car (Quest).
- Detachable parts: doors, hood, bumpers, mirrors. Windows crack, then shatter.
- Mechanical damage affects steering pull, engine power, and tire pressure. Burning cars persist as wrecks (section 7).

Other physics:

- Physics props in LOD0 only; sleeping bodies cost almost nothing.
- Destruction: pre-fractured meshes for glass, crates, drywall, and barriers, swapped in on hit. No runtime Voronoi on Quest.
- Ragdolls for NPCs with 11 bodies, settling to a static pose after 3 seconds.

11. Weapons, combat, damage

Weapons are physical objects handled with physics hands: real two-hand grips, manual reloads, and recoil that moves the hands rather than the camera. Gameplay values are data-driven and fictionalized for the 2040 setting.

Handling:

- Grip plus optional foregrip; with two hands, aim stabilizes and recoil recovery improves. Shouldering a long gun (stock near the face) adds further stability.
- Manual reload: eject the magazine, take a new one from a body slot, insert it, rack the slide or bolt. Each step is a typed grab point with a snap zone.
- Recoil is an impulse on the hand bodies (pitch, yaw, and push-back), never a camera kick, which avoids sickness.
- Sights: iron sights and red dots are rendered as simple reticle meshes on the weapon; magnified optics use a small render-to-texture at 256^2 on Quest, refreshed at half rate.
- Attachments are data: optics, grips, lights, suppressors, each changing stats and sound.

Ballistics (cheap and believable):

- Hitscan within 50 m; beyond that, projectile rays stepped per frame with drop and travel time.
- Material penetration table (glass, drywall, wood, car doors, concrete), so cover is honest.
- Impact decals and particles by surface, persisted per section 7.

Damage model:

- Player: body regions (head, torso, limbs) with armor plates; injuries slow movement or reduce aim stability; healing is a physical item applied with your hand.
- NPCs: locational damage, stagger, crawl when wounded, surrender behavior.

- No gore beyond blood decals and ragdolls; a toggle lowers this for younger or sensitive players.

Combat-aware generation:

- When combat odds rise (weapon drawn, on foot, recent fights), the generator increases cover objects, alternate routes, and vertical access in the next chunks, within the world logic of section 5.
- Each parcel exposes combat tags (cover density, sightline length, verticality, chokepoints, flank routes) so encounters are placed where the layout supports them.
- Encounter director: tension builds and releases on a curve; it never stacks fights back to back unless the player model says this player wants that.

12. NPCs, traffic, life sim

The world feels alive through tiered simulation: a few fully simulated NPCs near the player, many cheap ones further out, and statistical life beyond view. Quest caps at 24 full NPCs, 60 crowd agents, and 40 traffic cars on screen.

Simulation tiers:

Tier	Range	Cost per agent	Behavior
Full	0–30 m	~0.03 ms	Skinned mesh, navmesh pathing, perception, utility AI, ragdoll
Crowd	30–120 m	~0.002 ms	GPU-instanced vertex-animation (VAT) bodies on flow fields
Statistical	120 m+	~0	Density counts per chunk only; spawns agents on approach

NPC brains: utility AI scoring needs (work, home, eat, shop, socialize, flee, fight) against the time of day and district. Schedules come from the building they spawned from (residents leave home in the morning, workers enter offices). Combat NPCs add a cover-aware GOAP layer using the parcel combat tags.

Reactions: gunfire within 80 m makes pedestrians flee or hide and drivers abandon cars or speed off. Police response escalates by district and wealth. Witnesses remember the player for the session and a decay window across sessions.

Traffic:

- Lane graph generated with the road network; cars follow lane splines kinematically with intelligent driver model (IDM) spacing and simple lane changes.
- Intersections use signal phases or stop rules by road class.
- When hit or when the player is near and driving recklessly, a car becomes a full physics vehicle, then returns to kinematic once settled.
- Density follows time of day and district: rush hour on arterials, near-empty freeways at 4 a.m., parked cars dense in residential streets.

Variety: NPC appearance is modular (body, head, hair, clothing layers, colors) from a seeded combination, with region-appropriate clothing and climate. Fauna (birds, dogs, deer in forests, gulls on coasts) use the crowd tier.

Animation: full-tier NPCs use blend trees (locomotion by speed and direction, upper-body layers for carrying, phone, aiming) with foot IK on stairs and slopes, and head look-at toward the player when close. Crowd-tier NPCs use vertex animation textures with per-instance phase offsets so no two walk in sync. Motion capture source: CC0 or licensed mocap sets, retargeted once at build time. Avoid motion matching on Quest; consider it later for the PC tier.

13. Rendering, realism stack, day/night, weather

Quest cannot run path tracing or neural upscaling, so the look is faked with precomputed lighting, smart shaders, and cheap atmosphere. About 70% of the high-end look comes from lighting, wetness, and atmosphere, all of which can be faked. PC adds real screen-space effects on the same assets.

Lighting (solves baked GI vs time of day):

- Each chunk gets a grid of spherical-harmonic (L1) light probes, computed in the generation worker from sky visibility and sun occlusion on simplified geometry.
- Probes are baked for 4 lighting keys (dawn, noon, dusk, night) and blended by the world clock, so lighting moves with time without realtime GI.
- Kit pieces ship with baked ambient occlusion in vertex color or a second UV; chunk-level contact AO is baked into terrain.
- One realtime sun shadow cascade near the player; distant shading comes from baked AO and probe occlusion.

One uber shader (custom ShaderMaterial or onBeforeCompile on MeshStandardMaterial), with features toggled by define: detail normals, triplanar terrain, wetness, emissive, height fog, decal texture array, vertex-color AO, and wind sway for foliage. Fewer shader variants means fewer pipeline stalls on Quest.

Realism tricks (Quest-safe):

Effect	Technique
Wet roads and neon reflections	Global wetness drives roughness and a puddle mask; reflections from box-projected cubemaps per block, plus emissive reflection streak decals under signs
Neon glow	Emissive materials plus additive glow cards; no full-screen bloom on Quest
Glass towers	Box-projected cubemap reflections plus parallax interior mapping, so windows show fake rooms with depth for one texture lookup
Night windows	Per-window emissive lit by a seeded occupancy pattern that changes over the night
Atmosphere	Analytic height fog and aerial perspective in every shader; light-shaft cards; haze as in the Shenzhen references
Sky	Precomputed atmospheric scattering LUT, scrolling cloud layers, stars and moon
Car paint	Clearcoat shader: fresnel, cubemap, flake noise, dirt and damage masks
Water	One shader with per-biome color, turbidity, scrolling normals, cubemap reflection, foam at shores
Foliage	Alpha-tested cards and low-poly trunks, impostors beyond 120 m, wind in vertex shader
Detail at low resolution	Strong detail normals and decals, so surfaces read sharp under fixed foveation

Weather: the weather system drives wetness (0–1), fog density, wind, cloud cover, precipitation (rain or snow in a camera-attached particle cylinder), and puddle growth over time. Snow accumulation uses a world-up blend in the uber shader. Lightning flashes are a probe intensity spike plus a sky flash.

Day/night: 1 game day = 48 real minutes by default (configurable). Night is where the cyberpunk layer peaks: emissive density, signage animation, and traffic headlights.

PC tier additions: SSR, bloom, SSAO, TAA, 3 shadow cascades, 2× foliage and prop density, longer LOD reach, and higher texture tiers. Screen-space effects (SSR, bloom, SSAO, TAA) apply on PC Flat; in PC Streamed VR they depend on the XR post spike (section 3B).

Photoreal calibration:

- Physical light units: sun about 100,000 lux at noon, overcast about 10,000 lux, street lights about 10–30 lux on the ground, moonlight about 0.1 lux. Sky and sun come from the same atmosphere model.
- Exposure: physically based camera (EV100), with auto exposure computed from probe and sky luminance (no GPU readback), adapting over 1–3 seconds. Tunnels, interiors, and nights adapt like real eyes.
- Tonemapping: AgX (or ACES fitted), with per-biome and per-time color grading done in the material shaders via a small 3D LUT. White balance shifts warm at dusk and under sodium lights, cool under LEDs and moonlight.
- Linear workflow end to end: sRGB textures decoded to linear, lighting in linear, one output transform.

VR image stability (biggest realism win on Quest):

- MSAA 4x on the eye buffer; alpha-to-coverage for foliage and fences instead of plain alpha test.
- Specular anti-aliasing: roughness widened by normal-map variance and geometric curvature, so wet roads and car paint never sparkle.
- Normal map mips pre-filtered (Toksvig); texture LOD bias tuned for sharpness without shimmer.
- No geometry thinner than about 2 pixels at its LOD distance; thin wires and railings switch to textured cards.
- Dithered crossfade for every LOD and impostor switch; terrain geomorphs; nothing ever pops.

Night city lighting (thousands of lights for almost nothing):

- Street lights, shop fronts, and signs cast projected light pools as additive ground decals with soft falloff, plus a probe contribution. Only the 4 nearest lights are real dynamic lights.
- The player's car headlights are one real spot light with a projected light-cookie texture; traffic headlights are glow cards plus ground light pools.
- Wet surfaces stretch light pools into reflection streaks automatically from the wetness value.

PC tier realism additions: real-time reflections (SSR plus probes), contact shadows, volumetric fog, higher-resolution textures, denser foliage, and 3D clouds.

13A. Captured realism pipeline (how Quest reaches photoreal)

The Quest renders almost no lighting itself. Film-quality light is path-traced ahead of time by a cloud or home-PC baker running the same deterministic generator, and the headset displays the baked result plus a thin layer of real-time effects. This is the main rendering strategy of the game, and section 13's probe lighting becomes the fallback.

1. Cloud baker (the "alien tech" core):

- A GPU service (cloud instance, or the player's own PC as a home baker) runs the identical generator from section 5, so it builds the exact same chunk the Quest will build.
- It path-traces per chunk: lightmaps with full global illumination, bounced color, and soft shadows; box-projected reflection cubemaps per block; lit impostors for distant buildings; the sun-visibility shadowmask (section 3B); and potentially visible sets per 32 m cell.
- Time of day: 4 baked keys (dawn, noon, dusk, night) plus an overcast variant, blended in the shader between neighboring keys, with the real-time sun shadow and wetness layered on top.
- Encoding: RGBM lightmaps in KTX2 (UASTC), 4–8 texels per meter along roads and plazas, 1–2 elsewhere. Quest resident lightmap budget: 128 MB.
- Timing: the predictive agent requests chunks 30–120 seconds ahead; target bake time is under 2 seconds per chunk per key on an RTX-class GPU.
- Bake once, share forever: results are keyed by `(worldSeed, chunk, genHash, key)` and cached on a CDN, so any region baked once is free for every later visit and every player on that seed.

- Not baked yet or offline: section 13 probe lighting renders immediately, then crossfades to baked light when it arrives.
- Player changes: moved and dynamic objects use probes; a large change (a destroyed wall, a burned car) queues a rebake of that chunk while local probes cover the gap.

Baker technology (Claude builds it in tools/baker): default is the developer's own PC as a home baker (free); a cloud GPU is optional later for playing away from home. Stack: headless Chromium driven by Playwright, the same generator code, xatlas (WASM) for lightmap UV unwrapping, three-mesh-bvh for GPU ray traversal in shaders, a custom texel-space path tracer for lightmaps (each lightmap texel shoots rays against the BVH), and three-gpu-pathtracer for cubemaps and horizon panoramas. Output goes to the baked-data cache served by the backend (section 1A). Denoise lightmaps with a small edge-aware filter; verify bake quality with the reference-match test.

2. Baked horizon panoramas:

- For each region and time key, the baker renders 360° HDR panoramas of the generated far field from a grid of points, shown beyond 1–1.5 km. Distant skylines, mountains, and city lights are effectively photos of the world itself, which is why distance looks most real.
- Real HDRI skies (Poly Haven, CC0) or the atmosphere model fill the sky, matched to the time key.

3. Gaussian splats:

- Used for hero landmarks and for organic clutter that polygons fake badly: rock piles, bushes, rubble, trash heaps, tree clumps. Splat kits are captured once (the developer can scan real objects with phone apps such as Scaniverse or Polycam), then placed procedurally like any other prop.
- Rendered with a Three.js splat renderer (for example Spark or GaussianSplats3D; verify current versions), sorted in a worker.
- Quest budget: start at about 400k visible splats with low-order spherical harmonics and tune on device; PC runs full spherical harmonics and much higher counts.
- Limitation: splats carry the lighting they were captured in, so each kit is captured under neutral overcast light and tinted by the probe at runtime.

4. Photogrammetry kits:

- Facades, props, rocks, curbs, road surfaces, and vehicles come from scans: the developer's own captures plus CC0 libraries (Poly Haven, ambientCG). Check license terms before using Quixel Megascans outside Unreal.

- Quest meshes are 2–5k triangles per prop with normals and ambient occlusion baked down from the full-resolution scan; PC gets denser LODs.

5. What stays real-time on Quest: the near sun-shadow cascade, the player's headlights, muzzle flashes, wetness and puddles, particles, emissive signs, and moving objects lit by probes. These are layered over the baked image so the world still reacts.

6. PC Streamed tier: the same baked data at higher resolution, plus more real-time lights and shadow cascades, denser geometry and foliage, full-quality splats, and in-material volumetric fog; screen-space effects only where the XR post spike (section 3B) succeeds. It is rendered on the PC and streamed to the Quest, so the headset shows PC-class images.

14. Audio

Audio stays under 1 ms of main-thread time on Quest by pooling voices and spending HRTF only on the 8 nearest or loudest sources. Sound carries a large share of realism, so it gets the same care as visuals.

- **Voice pool:** 32 active voices max on Quest (64 on PC), prioritized by loudness × proximity × importance. HRTF panning for the top 8; equal-power panning for the rest.
- **Ambience beds:** per biome and district, layered by time of day and weather (city hum, distant traffic, freeway roar, forest birds and insects, surf, wind, rain on surfaces). Beds crossfade with the generator's district map.
- **Space-aware reverb:** 4 convolution impulse responses (open, street canyon, interior room, large hall or tunnel), selected from the space around the listener using a few raycasts per second.
- **Occlusion:** a low-pass filter on sources behind walls, from one raycast per voice every 100 ms.
- **Vehicles:** engine sound is granular, built from RPM-sampled loops crossfaded by RPM and load, plus tire squeal by slip, wind by speed, and suspension thumps.
- **Weapons:** layered close shot, mechanical detail, and distance tail, with the tail chosen by environment (street echo, open field, interior slap).
- **Persistence of sound:** places keep audio identity (a broken neon sign buzz, a dog that barks at the same house), seeded per chunk.
- **Music:** adaptive stems that react to driving speed, combat tension, and night, with a toggle for diegetic-only (car radio).

15. Core loop, progression, VR UI

The loop is: explore → discover jobs and places → earn money and reputation → upgrade vehicles, gear, and safehouses → reach further and stranger regions. Jobs are generated from the world's own logic, so they never feel bolted on.

Emergent jobs (generated from what exists nearby):

Job	Source in world logic	Playstyle served
Delivery and courier runs	Shops, warehouses, ports	Driving
Vehicle recovery	Scrapyards, towing yards, abandoned cars	Driving + light combat
Salvage in decayed buildings	Vacant factories, abandoned blocks	Exploration + interiors
Clear a contested site	Gang-held parcels with good combat tags	FPS
Photo and survey contracts	Landmarks, viewpoints, unexplored districts	Exploration
Escort and getaway	Banks, docks, police response	Driving + FPS

The bandit (section 8) learns which job types this player accepts and finishes, and offers more of those, while fatigue keeps variety.

Progression:

- Money buys vehicles, parts, weapons, attachments, and safehouses.
- Reputation per district unlocks better jobs, safer passage, or hostility.
- Safehouses (an apartment, a garage, a rural cabin) store vehicles and gear and act as save and fast-travel points. Fast travel only between discovered safehouses, with a fade.
- A discovered-world map fills in as you explore; generated places get generated names.

VR UI (diegetic first):

- Wrist device on the left arm: map, jobs, inventory, settings. Rendered as a WebXR quad layer so text stays sharp.
- Car dashboard shows navigation, speed, and job info; the radio is physical.

- Body slots for items (hips, chest, back) replace inventory menus for common gear.
- Health and damage shown through vignette, sound, and hand tremor, not a HUD bar. An optional minimal HUD exists for accessibility.
- Settings cover comfort (all locomotion options), seated or standing, handedness, subtitles, gore level, and quality tier.

15A. Living the life: body, character, phone, voices, law

These systems make the player a person in the world rather than a camera with hands. Each is a thin layer on the systemic simulation (9A), not a separate minigame.

Character and body:

- Character creator with real proportions: body type, face, skin, hair, age, scaled to the player's calibrated height so the IK body matches them.
- Body simulation, with toggles: stamina (sprinting, climbing, carrying), hunger and thirst, sleep (sleeping in a safehouse passes time), temperature by weather and clothing, and injuries that heal over time or with treatment. Default is light realism, with a hardcore option.
- Appearance persists: tattoos, haircuts (barbers), clothing bought or found, dirt, wetness, blood, bandages.

The phone (one device for most UI): calls and texts from NPCs and jobs, map and navigation, camera for photos stored in the save, flashlight, music, payments, and the job board. It is a physical object: pocketed, dropped, breakable, chargeable.

NPC voices and conversation:

- Barks and dialogue lines written by the AI Director (5B) become voice clips through an offline text-to-speech step on the baker or backend, cached per line, with simple viseme lip-sync. Voices vary by region, age, and character.
- Optional later: push-to-talk conversations with NPCs (speech-to-text on the backend, Director reply, TTS), with honest latency of 1–3 seconds. Off by default; fully playable without it.

Law and consequences: witnesses report crimes based on what they actually saw; police respond by district and severity; arrest means a fine, confiscation of illegal items, and a time skip; repeated crimes raise heat that decays over in-game days; vehicle and property damage cost money to repair. Good acts lower heat and raise reputation.

Onboarding: no tutorial screens. The first minutes are a lived scene in the cold-start region (8): wake in a safehouse, find the phone, go outside, find the car, drive. Each new interaction shows a short diegetic hint once, from the phone or a sticky note, then never again.

16. Debug, testing, failure modes

Every system ships with a way to see it, and every milestone passes an in-headset test before the next begins. The developer runs the headset tests and reports results; the model writes the checklists and the tools.

Debug tools (toggle from the wrist menu or a desktop overlay):

- Perf HUD: frame time, main-thread breakdown per system, draw calls, triangles, GPU memory estimate, worker queue depth, thermal quality step.
- Chunk visualizer: chunk states color-coded on a minimap and as ground overlays, predicted paths from the agent.
- Player model inspector: every feature (session and lifetime), fatigue per tag, bandit odds per decision, with sliders to override.
- Generator controls: force biome, district, time, weather; replay any chunk from its seed; diff a chunk's deltas.
- Free camera and time scale on desktop (Immersive Web Emulator).

In-headset test checklist (each milestone):

1. 10 minutes of the milestone's activity with OVR Metrics Tool: no frame time above 13.9 ms for more than 1% of frames.
2. 30-minute soak test for thermal throttling and memory growth (heap must stay flat).
3. Comfort check: no involuntary camera motion; locomotion options all work seated and standing.
4. Persistence check: change things, leave 1 km, return, quit, relaunch, return: identical.
5. Report results back as numbers plus notes on anything that felt wrong.

Failure modes and recovery:

Failure	Response
Generation falls behind (fast driving)	Raise fog density slightly, show LOD2 or impostors first, then refine; at worst, cap vehicle top speed softly via drag, never freeze
Memory pressure	Evict dormant chunks early, drop texture tier, reduce decal caps, flush saves
Frame time spikes	Thermal guard drops one quality step (section 4)
Teleport or fast travel	Fade out, generate the destination's 9 chunks, fade in
Save corruption	Fall back to the previous of 3 rotating slots; chunk deltas validated by checksum
Worker crash	Restart the worker, re-queue its jobs, keep the frame running
Player clips through world	Restore last safe position from a 2-second history ring

Automated tests (run on every build, desktop):

- Determinism: generate 500 random chunks twice and in shuffled order; their content hashes must match exactly.
- Coherence: the section 5B validator runs over 10 km² of generated world; failure rate must stay under 0.5%.
- Bot drive: an autopilot drives 20 km of freeway and city at 30 m/s while logging generation lag and frame time; any chunk not ready on arrival fails the build.
- Save round-trip: modify, save, reload, compare entity state byte for byte.
- Asset budget check: every texture, mesh, and material is checked against section 4 limits before it enters the build.

Reference-match test (the photorealism gate): for each biome and time of day, recreate the framing of one of the developer's reference photos in-game, then view the photo and the game side by side in the headset. A system does not ship until the developer judges it close to the photo. Log what differs (light, material, density, scale) and fix that first.

17. Milestones and scope

Build in this order; each milestone is playable on Quest 3S and passes the section 16 checklist before the next starts. The first three milestones prove the hardest risks: frame time, streaming, and the controller.

1. **M0 Foundation:** Vite + TS project, ECS, WebXR session, perf HUD, Rapier, worker pool, seeded RNG, IndexedDB save. Test: empty world at 72 Hz.
2. **M1 Road and car:** spline road modules (freeway and rural), terrain chunks streaming, hero car with VR wheel. Test: 10 minutes driving at 30 m/s with no hitches.
3. **M2 Controller:** physics hands, IK body, grab system, locomotion and comfort options, refactored from the existing controller. Test: comfort and latency in headset.
4. **M3 Persistence:** chunk records, deltas, decals, rotating saves. Test: section 16 persistence check.
5. **M4 World logic:** macro and meso generation, entity library v1, 2 city presets (US decay grid and Asian superblock), interchanges. Test: drive between districts.
6. **M5 Look:** uber shader, probes with time-of-day keys, wetness, fog, sky, glass interior mapping, neon. Test: frame time at night in dense city.
7. **M6 Learning:** player model, fatigue, bandit, predictive agent, cold-start region. Test: model inspector shows clear differences after 2 play styles.
8. **M7 Combat:** weapons, damage, combat tags, encounter director, enemy NPCs. Test: urban firefight at 72 Hz.
9. **M8 Life:** crowd and traffic tiers, NPC schedules, reactions. Test: busy street at rush hour.
10. **M9 Interiors:** interior grammar for apartment, shop, office, factory. Test: enter, fight, and leave without hitches.
11. **M10 Loop:** jobs, money, safehouses, wrist UI, map. Test: a 1-hour session that feels like a game.
12. **M11 Biomes and weather:** forest gorge, coast, snow, desert, rain and snow systems, connectors. Test: long drive across 3 biomes.
13. **M12 Polish and PC tier:** audio pass, PC effects, settings, accessibility.

v1 scope: everything above, single-player, Quest 3S and PC.

Later: multiplayer (shared or personal worlds), aircraft and boats, player building and base fortification, mod support for entity libraries and city presets, and a cloud baker for play away from the home PC.

Where the added systems land:

- M0 includes all section 3A rules (floating origin, pools, shader warmup, upload budget, deterministic math, service-worker caching), the automated determinism test, and the 3B spikes: multiview with MSAA, BatchedMesh on Quest Browser, splat renderer in XR, and reverse-Z support.
- M1 adds terrain erosion, CDLOD geomorphing, and road cut-and-fill (section 5A).
- M4 adds the coherence validator and region simulation (section 5B), plus the facade grammar and landmarks.
- M5 adds photoreal calibration, VR image stability, night light pools (section 13), and the texture ingestion pipeline (section 6A).
- M9 adds interactive interiors, elevators, portal culling, prop hydration, the uses system, shop inventories, and body persistence (section 9A); its test is the tattoo shop example.
- M10 adds the AI Director with offline template fallback, plus vehicle part assembly, tools, and repair (section 9A systemic simulation); its test is the crash and tire change example.
- M11 adds ecology-based vegetation and seasons, plus fire, water, electricity, and structural failure (section 9A).

Photoreal proof first (right after M0, before M1): build one 128 m city street chunk at dusk with cloud-baked lightmaps, reflection cubemaps, a horizon panorama, photogrammetry props, and one splat kit. It must pass the reference-match test against a developer photo and hold 72 Hz on Quest 3S. If it fails, fix the rendering approach before building anything else, because the whole game depends on this look.

17A. Release and store readiness

Shipping is part of the build, not an afterthought, so the game can reach the Meta Horizon Store and the web without a scramble at the end.

- **CI:** GitHub Actions runs typecheck, tests, bench, and screenshot diffs on every push; a failing budget blocks the merge.
- **Hosting:** static files on a CDN plus the small backend (1A); COOP and COEP headers enabled so SharedArrayBuffer works; HTTPS everywhere.
- **Packaging:** PWA manifest, service worker, versioned asset bundles with cache busting; Quest packaging through Meta's PWA tooling in Meta Quest Developer Hub; the web build is the same files.

- **PC Streamed setup:** Chrome on the PC with the OpenXR runtime set to Meta Link, Air Link, or Virtual Desktop; the game detects the tier and loads PC budgets.
- **Store requirements:** comfort rating (all locomotion options present), age rating for violence, a privacy policy covering the AI Director and the optional voice features, data-safety declarations, and an offline mode statement.
- **Versioning:** semantic versions tagged per milestone; save files carry their version and migrate forward; old visited chunks never regenerate differently (section 7).
- **Production telemetry:** opt-in only, anonymous frame-time and crash reports to the backend; never the player model.
- **Support loop:** a bug report button in the release build (same as the dev Report button, anonymized) so real players feed the same fix loop Claude Code already uses.

18. Output requirements

Claude Code starts with M0, then the photoreal proof slice, and produces working, committed code, never a design essay. Each task delivers one vertical slice that runs, following the 1A work loop. First session: scaffold the repo and CLAUDE.md, the dev server with the headset bridge, the test and bench runners, and an empty WebXR scene at 72 Hz with the perf HUD.

Every response must include:

1. Files with full paths in the project structure (`src/core` , `src/xr` , `src/gen` , `src/world` , `src/render` , `src/physics` , `src/ai` , `src/audio` , `src/ui` , `src/save` , `workers/`).
2. The data schemas the code reads and writes, versioned.
3. Measured or estimated Quest 3S cost per system, and how the PC tier scales it.
4. An in-headset test checklist for that milestone (section 16 format).
5. Known shortcuts or placeholders, listed explicitly.

Inputs the developer will supply over time: in-headset test results, the existing VR controller code, texture and photo references, Google Maps street and aerial screenshots, and decisions on anything marked open. Ask only for these.

Open risks to prototype early:

Risk	Why it is hard	Early prototype
Streaming at 30 m/s on Quest	Generation, meshing, and upload all compete with the frame	M1 freeway drive with worker timing on HUD
Physics hands latency	Main-thread Rapier plus IK must stay under 2.5 ms total	M2 hands-only test scene with 50 props
Probe-baked time of day	Worker probe baking cost per chunk	Bake 1 chunk, 4 keys, time it in the worker
Thermal throttling	Sustained load drops clocks after minutes	30-minute soak in M1 and M5
Interior memory	Many interiors resident in dense cities	On-demand interiors with a 3-interior cap on Quest
Learning feels real	Bias must be noticeable but not weird	Inspector A/B: two players, same seed, compare after 20 min